

PointISA: ISA-Extensions for Efficient Point Cloud Analytics via Architecture and Algorithm Co-Design

Meng Han
School of Integrated Circuits
Tsinghua University
Beijing, China
mhan@mail.tsinghua.edu.cn

Hao Zhang
State Key Laboratory of CCSE
Beihang University
Beijing, China
zh19373126@buaa.edu.cn

Jianfeng Zhu
School of Integrated Circuits
Tsinghua University
Beijing, China
jfzhu@tsinghua.edu.cn

Liang Wang^{*}
State Key Laboratory of CCSE
Beihang University
Beijing, China
lwang20@buaa.edu.cn

Bowen Jiang
State Key Laboratory of CCSE
Beihang University
Beijing, China
jiangbw@buaa.edu.cn

Shaojun Wei
School of Integrated Circuits
Tsinghua University
Beijing, China
wsj@tsinghua.edu.cn

Limin Xiao
State Key Laboratory of CCSE
Beihang University
Beijing, China
xiaolm@buaa.edu.cn

Xilong Xie
State Key Laboratory of CCSE
Beihang University
Beijing, China
xxl1399@buaa.edu.cn

Leibo Liu^{*}
School of Integrated Circuits
Tsinghua University
Beijing, China
liulb@tsinghua.edu.cn

Abstract

Point cloud analytics plays a crucial role in spatial machine vision for applications like autonomous driving, robotics and AR/VR. Recently, numerous domain-specific accelerators have been proposed to meet the stringent real-time and energy-efficiency demands of embedded and mobile platforms. However, most existing point cloud accelerators are kernel-independent hardware architectures, which face significant limitations and struggle to adapt to the rapidly evolving nature of point cloud applications.

To address these challenges, we propose PointISA, a novel Instruction Set Architecture (ISA) extension designed for efficient point cloud analytics through architecture and algorithm co-design. First, we incorporate specialized instructions into PointISA for point cloud processing, such as Euclidean distance computation and multidimensional sorting. And then, we present a unified hardware architecture that efficiently executes both PointISA's specialized instructions and conventional matrix multiplication. Finally, we optimize key algorithms like farthest point sampling and k-nearest neighbors, adapting them into parallel multiple-points-to-multiple-points (MP2MP) patterns that fully exploit the architecture's parallel capabilities while maintaining algorithmic correctness. Evaluation shows that PointISA achieves an average 5.4× speedup and 4.9× improvement in power efficiency across diverse point cloud workloads while incurring a negligible area overhead of 0.9%.

^{*}Corresponding Authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/25/10

<https://doi.org/10.1145/3725843.3756122>

Keywords

Point cloud Analytics, ISA-Extension, Architecture and Algorithm Co-Design

ACM Reference Format:

Meng Han, Liang Wang, Limin Xiao, Hao Zhang, Bowen Jiang, Xilong Xie, Jianfeng Zhu, Shaojun Wei, and Leibo Liu. 2025. PointISA: ISA-Extensions for Efficient Point Cloud Analytics via Architecture and Algorithm Co-Design. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3725843.3756122>

1 Introduction

The rapid advancement of spatial machine vision technologies has made point cloud analytics an essential computational workload, particularly for battery-powered embedded and mobile platforms that requiring high efficiency and real-time processing. This computational demand spans a wide range of application domains, including autonomous driving[21], robotics[30] and virtual/augmented reality (VR/AR)[50]. Unlike structured 2D image data, a point cloud is an unordered and sparse set of points, directly representing a physical object or three-dimensional scene without inherent neighborhood information. These intrinsic properties result in fundamentally different characteristics compared to traditional convolutional neural network (CNN) and transformer-based workloads[26]. Consequently, numerous domain-specific accelerators have been proposed, aiming to satisfy the stringent real-time and energy-efficiency requirements of embedded and mobile platforms[5, 11–13, 16–18, 32, 45, 47].

Most existing point cloud accelerators fall under the category of **kernel-independent hardware architectures**. They integrate

multiple specialized accelerators, each tailored for specific kernels¹, to address the diverse requirements of point cloud processing pipelines. For instance, PointAcc[26] features a mapping unit for point cloud downsampling and neighbor search kernels, alongside a systolic array[20] for matrix multiplication. Although there exist considerable shared computational patterns among these kernels, the coarse-grained nature of such heterogeneous integration often results in only partial activation of hardware modules during execution, leading to suboptimal computational utilization. Our case study reveals that the average computational utilization of PointAcc in DGCNN[41] workload is below 1.7% (detailed discussion in Section 3.1), severely limiting its efficiency in supporting point cloud analytics. Furthermore, the kernel-independent hardware architectures struggle to adapt to the rapidly evolving point cloud applications. A recent point cloud accelerator, Crescent[10], for example, primarily focuses on optimizing geometric nearest neighbor search in point cloud workload using a custom KD-tree based approach. While effective for low-dimensional spatial queries, it fails to address the high-dimensional feature space searches that are increasingly common in modern point cloud neural networks[41]. In high-dimensional spaces, the performance of KD-tree based methods often degrades due to the curse of dimensionality, sometimes underperforming brute-force search[7].

Kernel-independent hardware architectures face significant limitations in point cloud processing, prompting us to propose a novel approach using Instruction Set Architecture (ISA) extensions within the CPU. Kernel-independent hardware architectures perform well for CNNs and Transformers, which benefit from dense and uniform computational patterns. However, point clouds present diverse workloads. This diversity poses significant challenges to the efficiency and scalability of such architectures, particularly for rapidly evolving point cloud applications. To address these limitations, we reconsider the technical approaches for point cloud accelerators and explore accelerating point cloud applications within the CPU through ISA extensions. The ISA-extension approach presents several inherent advantages tailored to the demands of point cloud processing. By integrating specialized instructions directly into the CPU, it eliminates the costly data transfers often required between host and out-of-core accelerators. Additionally, it leverages the CPU's robust control flow capabilities to effectively manage the diverse pre- and post-processing tasks inherent in point cloud algorithms. Furthermore, ISA extensions provide greater flexibility, enabling adaptation to emerging point cloud techniques without necessitating extensive hardware redesigns. However, simply translating the kernels from existing kernel-independent hardware architectures into isolated instructions within the CPU is insufficient for efficient point cloud processing and may even result in negative performance due to the instruction-driven execution model of CPUs. Instead, we advocate for a unified structure within the CPU to support the diverse kernels of point cloud workloads, aiming for more adaptive architectures that can efficiently support the rapidly evolving point cloud workloads.

To address these challenges, we propose PointISA, a novel instruction set architecture extension designed for efficient point

cloud analytics through architecture and algorithm co-design. First, PointISA strategically divides point cloud neural network workloads into computation-intensive and control-intensive components, executing the former on a highly parallel coprocessor driven by specialized instructions (e.g., for Euclidean distance computation and multidimensional sorting) and the latter on a traditional out-of-order pipeline with algorithmic co-design to simplify control logic. This separation allows PointISA to support a wide range of point cloud workloads with high efficiency and low hardware cost. Such lightweight design is especially valuable in embedded systems with strict area and power constraints. Then, we design a lightweight and versatile systolic array architecture that efficiently executes the PointISA's specialized instructions and conventional matrix multiplication. Finally, through hardware-software co-design, we introduce optimized multiple-points-to-multiple-points (MP2MP) computation patterns for critical kernels like Farthest Point Sampling (FPS) and k-Nearest Neighbors (kNN), fully exploiting the architecture's parallel capabilities while maintaining algorithmic correctness.

We validate our approach by extending ARM's AArch64 ISA in the gem5[3] simulator and integrating the new instruction into the LLVM[23] compiler. The evaluation results demonstrate that PointISA achieves, on average, a 5.4× speedup and a 4.9× improvement in power efficiency compared to baseline implementations, while incurring a negligible area overhead of just 0.9%.

In summary, this paper presents the following contributions.

- We identify the limitations of prevalent kernel-independent accelerator architectures in efficiently handling the diverse and evolving nature of point cloud workloads, highlighting the resulting inefficiencies and motivating the exploration of ISA extensions as a novel optimization direction.
- We design and propose PointISA, a novel CPU ISA extension framework specifically targeting efficient and adaptable point cloud analytics, suitable for integration into modern processors, particularly those for embedded AI systems.
- We propose a unified hardware architecture for PointISA that leverages systolic array structures. This design maximizes computational resource reuse and achieves significant acceleration with only a negligible 0.9% area overhead.
- We introduce algorithm-architecture co-design techniques, including novel transformations for key kernels like FPS and kNN, to restructure computations into highly parallel patterns that maximize the utilization and efficiency of the PointISA hardware implementation.

2 Background

We first introduce point clouds and their applications in Section 2.1. Then, we describe fundamental point cloud operations, including farthest point sampling and neighbor search in Section 2.2. Finally, we provide an overview of the Out-of-Order CPU pipeline and the CPUs' matrix extensions in Section 2.3.

2.1 Point Cloud-based Applications

A point cloud is a collection of points $P = \{p_i\}$, where each point p_i is represented by its Cartesian coordinates (x, y, z) . Point clouds are are typically generated by 3D sensing technologies, such as LiDAR

¹In this paper, a kernel refers to a fundamental algorithm tailored for a specific task within the point cloud processing pipeline, such as downsampling, neighbor search or matrix multiplication.

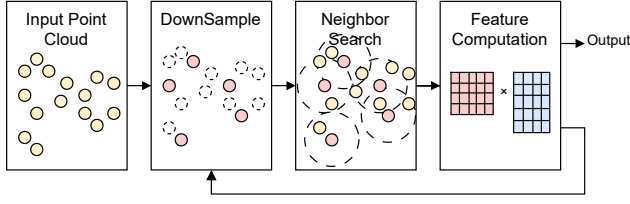


Figure 1: An example of deep learning-based point cloud analytics pipeline.

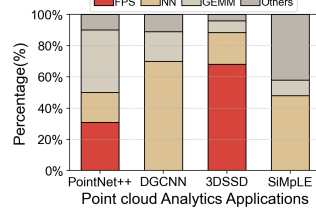


Figure 2: Latency breakdown for PointNet++[35], CPU core with matrix extension, DGCNN[41], 3DSSD[44] and SiMPL[2].

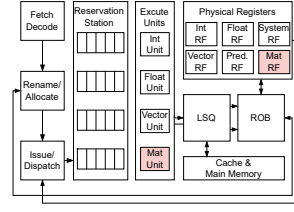


Figure 3: An Out of Order execution. Blocks in red are the main components for matrix extension.

and depth cameras, to capture detailed spatial information about the physical environment[37]. With the increasing affordability and miniaturization of these sensors, point clouds have become integral to battery-constrained embedded and mobile platforms, supporting a wide range of applications, including autonomous vehicles[21], robotics[30] and AR/VR[50].

Unlike structured image data, where neighborhood information is readily accessible through pixel indices, point clouds are inherently unstructured and permutation-invariant[34]. This irregularity prevents the direct application of conventional feature extraction techniques, such as convolutional filters or multilayer perceptrons (MLPs)[33], complicating the extraction of meaningful representations. To address this challenge, point cloud neural networks employ specialized mapping kernels to reorganize and structure the data for effective processing.

As depicted in Figure 1, the typical point cloud processing pipeline begins with downsampling, which selects a representative subset of points. Among various downsampling techniques, farthest point sampling is widely adopted for its ability to maintain geometric fidelity. Following this, a neighbor search operation identifies local neighborhoods around each sampled point. These sampled points and their neighbors are then aggregated into a structured 2D matrix to enable subsequent feature computation, often through matrix multiplication operations. Together, these stages form a macro-layer, which is iteratively applied within point cloud neural network models to support diverse spatial machine vision tasks, such as classification, object detection and segmentation[15].

2.2 Diverse Point Cloud Kernels

Point cloud applications encompass a variety of computational kernels. To identify opportunities for targeted acceleration, we profiled

four representative point cloud applications—three point cloud neural networks (PointNet++[34], DGCNN[41], 3DSSD[44]) and one point cloud registration method (SiMPL[2]) on an edge SoC platform (NVIDIA AGX Xavier). As illustrated in Figure 2, the analysis reveals that farthest point sampling (FPS), neighbor search (NN) and matrix multiplication are the primary computational bottlenecks. While point cloud analytics pipelines also incorporate other operations, such as Morton/Hilbert curve encoding and various filtering techniques, our profiling indicates that their latency contributions are comparatively minor. Notably, FPS and NN alone account for 50% to 88% of the end-to-end inference latency. Consequently, this paper focuses on optimizing these critical kernels.

2.2.1 Farthest point sampling. Farthest point sampling is a fundamental operation for point cloud downsampling, effectively reducing point density while preserving the underlying geometric structure. As a core component of modern point cloud processing pipelines, FPS is widely adopted in various point cloud neural networks[39, 49]. FPS generates a subset of the point cloud iteratively by selecting the point that is farthest from the previous sampled points in each iteration. Generally, the FPS consists of two principal operations per iteration: 1) *Distance computation*: Calculating the minimum distance from each candidate point to existing samples. 2) *Point selection*: Identifying the candidate with the maximum distance.

Variants and Extensions: While traditional FPS operates exclusively in 3D geometric space, recent advancements have introduced feature-space FPS, which processes high-dimensional feature representations [44]. The key distinction lies in the use of high-dimensional Euclidean distances to identify the farthest point. In contemporary architectures, hybrid approaches often combine geometric and feature-space FPS to leverage their complementary strengths.

Execution Methods: Currently, FPS can be implemented using two primary computational approaches, each with distinct characteristics. The brute-force method computes distances between all candidate points and the existing sample set during each iteration, resulting in a computational complexity of $O(nk)$ algorithm complexity, where n is the number of points and k is the size of the sample set. This approach is applicable to both 3D geometric and high-dimensional feature spaces, but it incurs significant computational overhead due to the exhaustive nature of the calculations. Alternatively, tree-based structures, such as KD-trees or octrees, accelerate the process by partitioning the point cloud into hierarchical spatial regions, reducing unnecessary computations [17, 18] and achieving a lower complexity of $O(n \log n)$. However, these methods can only apply to 3D geometric space and introduce control-intensive operations due to tree traversal and dynamic branching, which are challenging to parallelize and optimize on traditional accelerators.

2.2.2 Neighbor Search. Neighbor search is a fundamental operation in point cloud processing, enabling local feature aggregation and geometric relationship modeling. Modern point cloud neural networks primarily employ two variants of this operation: k-nearest neighbors search and radius search. The process consists of two main steps: 1) *Distance computation*: Calculating pairwise distances

between query points and candidates. 2) Neighbor selection: Identifying relevant neighbors based on specified criteria, such as the top k nearest points or points within a given radius threshold.

Variants and Extensions: While traditional neighbor search operates in 3D geometric space, modern approaches have extended this operation to high-dimensional feature spaces [41]. In practice, hybrid methods often combine geometric and feature-based neighbor search to exploit their complementary advantages. The feature-space variant is particularly valuable in later network layers, where semantic information becomes more critical than pure geometric relationships.

Execution Methods: Similar to FPS, neighbor search can be executed via brute-force or tree-based approaches. The brute-force method computes distances between the query point and all candidates, leading to a computational complexity of $O(n)$ per query, which becomes a significant bottleneck for large point clouds. Tree-based methods, such as KD-trees or octrees[5], mitigate this by organizing points into spatial hierarchies, enabling faster queries. However, these approaches involve control-intensive operations due to tree construction and traversal, which often result in irregular memory access patterns and limited parallelism, posing challenges for hardware acceleration.

2.3 CPU Pipeline Structure and Matrix Extensions

Recent advancements in CPU architectures have increasingly focused on integrating specialized computation units and instruction set extensions to accelerate AI workloads. Major architectures, such as Intel’s Advanced Matrix Extensions (AMX)[22] and ARM’s Scalable Matrix Extension (SME)[42], introduce dedicated matrix operation instructions tightly coupled with the CPU’s out-of-order execution pipeline. These extensions typically employ systolic array structures to efficiently perform matrix operations with high throughput[19]. As illustrated in Figure 3, matrix processing units and associated registers enable fine-grained integration of specialized instructions with existing CPU functionalities. This architectural approach supports high instruction-level parallelism and flexible interaction with matrix registers, offering significant advantages for handling complex control patterns in workloads like point cloud processing, particularly for kernels such as farthest point sampling and neighbor search.

However, while these matrix extensions are optimized for AI tasks involving general matrix multiplication (GEMM), they fall short in addressing the unique computational demands of point cloud kernels. Operations in FPS and NN predominantly rely on Euclidean distance calculations, which are not efficiently supported by existing matrix multiplication instructions. This mismatch highlights a critical gap in current CPU extensions, underscoring the need for tailored architectural solutions to effectively accelerate the diverse computational patterns inherent in point cloud analytics.

3 Motivation

To address the challenges in point cloud analytics, we analyze the limitations of existing accelerators through three key observations.

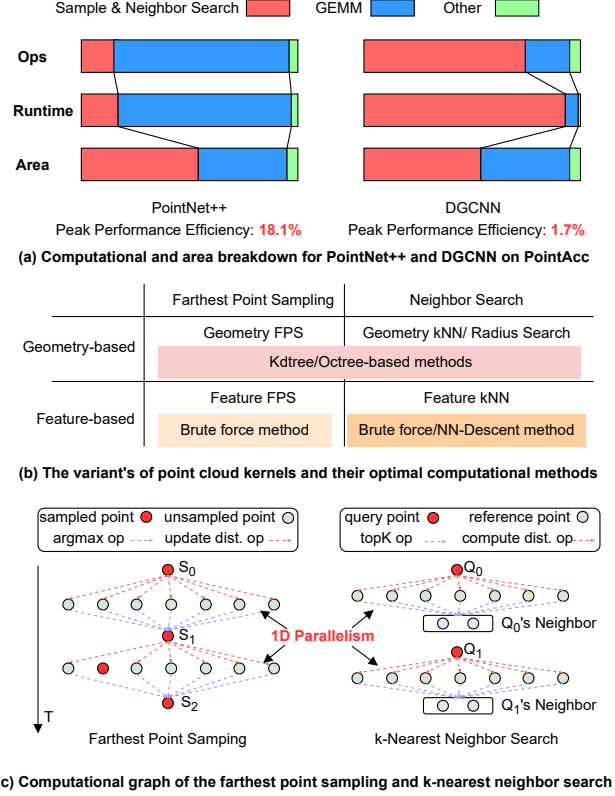


Figure 4: Three observations of point cloud workloads.

This section focuses on the inefficiencies stemming from architectural and workload characteristics, providing the motivation for our proposed solution.

3.1 Observation and Key idea

Through an extensive exploration of existing point cloud accelerators, we have three crucial observations that demonstrate the substantial space for point cloud analytics improvement, which is shown in Figure 4.

Observation I: The kernel-independent hardware architectures result in low computational utilization under imbalanced workloads.

Figure 4(a) illustrates the distribution of computational operations (Ops), execution time and area allocation of computation units in PointAcc, a representative point cloud accelerator, under PointNet++[35] and DGCNN[41] workloads on the S3DIS dataset. PointAcc primarily consists of Mapping Units for farthest point sampling and neighbor search kernel using a brute-force method (executed via a SIMD architecture) and systolic arrays for matrix multiplication.

For PointNet++, the computational demand of sampling and neighbor search operations is significantly lower than that of feature computation, leading to shorter execution times for these tasks. However, the Mapping Units occupy a substantial portion of the accelerator’s area, resulting in approximately half of the hardware

remaining idle during most of the execution. Consequently, the end-to-end utilization is limited to just 18.1%. The DGCNN workload exhibits an even more pronounced imbalance, with neighbor search operations dominating over 80% of the total execution time. This causes prolonged idleness in the systolic arrays, further reducing the average utilization to 1.7%. Even under ideal conditions with unlimited on-chip memory and bandwidth, PointAcc’s theoretical optimal utilization is only 23.6%. The rigid allocation of hardware resources fails to adapt to the dynamic and uneven computational demands of point cloud workloads. Such low resource utilization highlights the urgent need for a more unified and adaptive architectural design.

Key idea: We propose a versatile systolic array structure capable of executing matrix multiplication as well as operations such as Euclidean distance computation and sorting within a unified hardware framework. This approach leverages the arithmetic units and dataflow originally designated for matrix multiplication to perform Euclidean distance calculations and sorting, significantly reducing area overhead.

Observation II: Diverse point cloud kernels exhibit varied execution preferences without a one-size-fits-all solution.

As discussed in Section 2.2, point cloud applications encompass a variety of computational kernels, each with distinct execution characteristics and resource demands. Figure 4(b) illustrates the performance of optimal neighbor search and farthest point sampling algorithms across geometric (low-dimensional) and feature-space (high-dimensional) dimensions, reveals significant variations in optimal execution strategies depending on the kernel type and data dimensionality.

For instance, in low-dimensional geometric spaces (e.g., 3D spatial coordinates), the tree-based methods (e.g., KD-trees or octrees) for FPS and NN often outperform brute-force approaches due to their ability to efficiently partition the space, reducing computational complexity to $O(n \log n)$ compared to the $O(n^2)$ complexity of brute-force methods. Existing accelerators, such as QuickFPS[18] and Crescent[10], design customized hardware to efficiently traverse, backtrack and prune KD-trees, significantly enhancing computational efficiency. However, in high-dimensional feature spaces (e.g., 64 or more dimensions), the effectiveness of tree-based methods diminishes due to the “curse of dimensionality,” where spatial partitioning becomes less efficient, and effective pruning strategies deteriorate. This often results in frequent, irregular memory accesses that further degrade performance, making brute-force methods preferable despite their higher theoretical complexity. These disparities indicate that no single execution method—whether brute-force, tree-based, or array-optimized—can achieve optimal performance across all kernels and dimensional contexts, highlighting the lack of a one-size-fits-all solution.

Designing an accelerator to support multiple execution modes introduces significant challenges in control logic complexity. For example, QuickFPS[18] is a custom accelerator designed specifically for KD-tree-based FPS, implementing a 4-stage pipeline for tree traversal and pruning. Control circuitry in QuickFPS accounts for 47.6% of the total accelerator area. When using QuickFPS for feature-space FPS operations (where brute-force methods are preferred), this extensive control logic becomes essentially unused overhead. Furthermore, extending such an accelerator to support

additional algorithms like KD-tree-based kNN would require even more complex control pathways, making a comprehensive solution impractical in terms of hardware efficiency and design complexity.

Key idea: We identify common computational patterns across diverse kernels and decomposing their execution modes into distinct components. Specifically, we extract compute-intensive operations—such as Euclidean distance calculations and matrix multiplications—into specialized instruction extensions that can be efficiently executed on a unified hardware framework. Meanwhile, control-intensive operations—such as tree traversal and dynamic branching—are offloaded to the CPU’s flexible pipeline for adaptive execution. This hybrid approach leverages the strengths of both specialized hardware and general-purpose processing, enabling dynamic adaptation to varying kernel requirements without sacrificing performance or efficiency.

Observation III: Mismatched parallelism dimensions among point cloud kernels hinder unified hardware acceleration

Point cloud processing involves a variety of computational kernels with inherently different parallelism dimensions, posing significant challenges to designing unified hardware accelerators. Feature computation often shows high parallelism. It benefits from processing multiple points at once. In contrast, current state-of-the-art implementations[38] of operations like FPS and kNN use single-point-to-multiple-point (SP2MP) distance calculations. Their parallelism is limited by the sequential or localized nature of these tasks. Even when a unified hardware structure, such as a systolic array, is designed to support diverse point cloud operations, FPS and kNN workloads can only be mapped to a small fraction of the available computational units, leading to severe underutilization.

Figure 4(c) illustrates the computational patterns within FPS and kNN kernels, highlighting the inherent limitations in parallelism exploitation. In FPS, only one sample point is generated per iteration. To identify the next farthest point, the current implementation computes distances between the latest sampled point and all or a subset of unsampled points, result in a SP2MP manner. Similarly, for each query point, kNN computes the distance to multiple reference points in SP2MP manner. As a result, this mismatch in parallelism dimensions exacerbates the computational utilization problem. This highlights the need for a software-hardware co-design approach that adjusts the algorithm’s instruction flow to better align with and exploit the parallelism provided by the hardware, thereby improving computational efficiency.

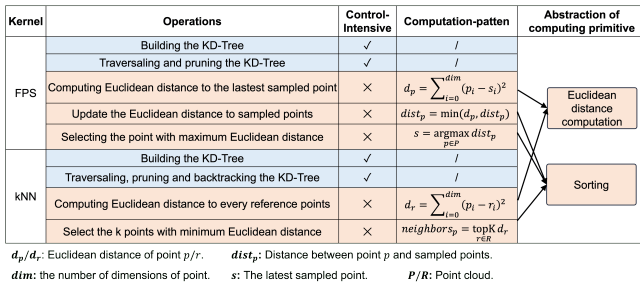
Key idea: We propose efficient algorithmic optimizations for farthest point sampling and neighbor search through hardware-software co-design. By identifying and delaying noncritical computations without compromising accuracy, we extract two-dimensional parallel features to generate MP2MP computational tasks. This approach fully leverages the capabilities of our proposed ISA, maximizing resource utilization across diverse workloads.

3.2 Design Consideration

In this paper, we propose PointISA, a novel instruction set architecture (ISA) extension designed to accelerate point cloud analytics through hardware-software co-design. By analyzing diverse computational workloads, we identify essential operations and design a unified hardware structure to support them efficiently at the

Table 1: Comparison with exist accelerators.

	Distance Computation	Parallelism Degree	Architecture	Computational Accuracy	Flexibility	Operational Intensity
PointAcc[26]	Point-wise	1D	Kernel-independent architecture Mapping Unit + Systolic Array	Exact	Medium	Low
Crescent[10]	Point-wise	1D	Kernel-independent architecture KD tree Pipeline + Systolic Array	Approximate	Low	Low
FusionArch[31]	Point-wise	1D	Kernel-independent architecture FPS Engine + kNN Engine + Cluster Engine + Computing Engine	Exact	Low	Medium
GDPCA[4]	Point-wise	1D	Kernel-independent architecture NN Engine + Update Engine + Aggregate Engine	Approximate	Medium	Medium
This work	Dimension-wise	2D	ISA-Extension in CPU Versatile Systolic Array	Exact	High	High

**Figure 5: Decomposition and Abstraction of Point Cloud Analytics Kernels.**

ISA level. To fully leverage the fine-grained parallelism offered by PointISA, we restructure key algorithms such as farthest point sampling and nearest neighbor search. Specifically, we introduce a lazy computation-based technique that transforms inherently sequential workloads into two-dimensional parallel tasks, enabling end-to-end acceleration across diverse point cloud applications.

To the best of our knowledge, this is the first work to optimize diverse point cloud workloads through ISA extensions. Unlike prior efforts that focus on isolated kernel accelerations or fixed hardware designs, our methodology provides a comprehensive and adaptable solution. A detailed comparison with existing work is presented in Table 1, highlighting the advantages of our proposed framework in terms of flexibility, efficiency and scalability.

4 ISA-Extension for Point Cloud Analytics

4.1 Design Principles

The design of PointISA is guided by the principle of decoupled specialization: computation-intensive operations are offloaded to custom instructions that exploit parallelism in hardware, while control-intensive logic remains on the CPU’s out-of-order pipeline for flexibility. This approach addresses the computational challenges of point cloud analytics, particularly in managing the complex interplay between compute-intensive calculations and data-dependent control flow in operations like FPS and NN. By carefully

partitioning workloads between specialized hardware and general-purpose cores, PointISA effectively accelerates performance-critical components while maintaining programming flexibility.

Separation of Computation- and Control-Intensive Operations: As shown in Figure 5, we identify computation-intensive tasks, such as Euclidean distance calculations and sorting, exhibit high parallelism and can be efficiently accelerated through custom ISA instructions. In contrast, control-intensive operations, like KD-tree node traversal and pruning, involve irregular control flow and low parallelism, making them better suited for the CPU’s out-of-order execution pipeline. This separation ensures that hardware resources are utilized effectively, avoiding the inefficiencies of forcing all operations into a rigid parallel framework.

Abstraction of Common Computational Primitives: By decomposing the FPS and NN kernels (as illustrated in Figure 5) into fine-grained computational patterns, we identify two dominant classes of primitives: (1) Euclidean distance computation, and (2) distance-based aggregations, such as *argmax*, *min*, and *top-K*. All of these aggregation operations can be reduced to a generalized sorting primitive. Based on this decomposition, we abstract the core functionality of diverse point cloud analytics into a small set of reusable instructions, thus improving adaptability. While our evaluation focuses on FPS and kNN, the flexibility of PointISA extends well beyond these kernels. The same instruction primitives can readily support other important point cloud operations such as k-means clustering in point cloud filtering[9] and Morton encoding in point cloud compression workloads[46]. This is because these operations rely on the same fundamental computations—distance calculations and subsequent data reorganization—that our instruction set is designed to accelerate.

Dimension-Agnostic Support with Flexibility: Since Euclidean distance calculations in point cloud kernels often involve dynamic dimensions(e.g., low-dimensional geometric spaces or high-dimensional feature spaces), PointISA instructions are designed to handle dimension-wise processing. This allows efficient execution in both geometric and feature spaces, while the CPU manages any dimension-dependent control logic, ensuring scalability without compromising performance.

Minimization of Data Movement Overhead: By fusing computation and aggregation operations at the ISA level, PointISA reduces unnecessary data transfers between registers, cache and

Table 2: The proposed extension instructions for point cloud analytics

Instruction		Description
Comp.	SEDS Ta, Pn, Pm, Zn, Zm	Squared Euclidean Distance between two Single-float vector registers - Computes the pairwise squared Euclidean distances between elements of vector registers Zn and Zm, with masking controlled by predicate registers Pn (for Zn) and Pm (for Zm). The results are accumulated into the matrix tile Ta.
	HSTWI Ta, Tb, Pn, Pm	Horizontal Sort two Tiles with Index Tiles- Performs horizontal (row-wise) sorting of elements across matrix tile registers Ta and Tb, where each row in Ta and Tb is sorted in ascending order . The sorting process is mask-controlled by predicate registers Pn (for Ta) and Pm (for Tb), which determine whether each element participates in the operation. Two implicit index matrix registers (Ta+8 and Tb+8) track the original positions of the sorted elements, preserving their indices for later reference.
Sort	VSTWI Ta, Tb, Pn, Pm	Vertical Sort two Tiles with Index Tiles - Analogous to HSTWI but operating on columns .
	USTWI Ta, Tb, Xn, Xm	Unified Sort tow Tiles with Index Tiles - Performs a unified masked sort across all elements of matrix tile registers Ta and Tb, treating them as a single contiguous array. The number of active elements to sort in Ta and Tb are specified by the general-purpose registers Xn (for Ta) and Xm (for Tb). Implicit index matrices (Ta+8 and Tb+8) track original positions during sorting, preserving permutation metadata.
Move	MOVT Ta, Tb, Pn, Pm	Move one Tile to another - Performs a predicated move of data from matrix tile register Ta to Tb, where the transfer is controlled element-wise by predicate registers Pn (for Ta) and Pm (for Tb). Only elements with active mask bits are copied; others in Tb remain unchanged.

• SEDS Ta, Pn, Pm, Zn, Zm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	1	0	0				Zm		Pm		Pn				Zn				0	1		Ta	0

• HSTWI Ta, Tb, Pn, Pm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1		Ta		Pm		Pn		0	1		Tb		0	0	0	0	0	0

• VSTWI Ta, Tb, Pn, Pm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1		Ta		Pm		Pn		1	1		Tb		0	0	0	0	0	0

• USTWI Ta, Tb, Xn, Xm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	0	0	1	1	Ta		Tb		Xn					Xm					1	1	0		

• MOVT Ta, Tb, Pn, Pm

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	0	0	1	0		Ta		Pm		Pn		0				Tb		0	1	0	0	0	0

Figure 6: The instruction formats of PointISA. Ta/Tb denote matrix tile registers, Pn/Pm denote predicate registers, Zn/Zm denote vector registers and Xn/Xm denote general-purpose registers.

memory. This is particularly beneficial for point cloud workloads, where frequent data access can lead to bottlenecks, and it complements the CPU’s ability to handle control flow dynamically.

4.2 Proposed Instruction Set

Table 2 provides an overview of the PointISA instruction set, which comprises five custom instructions optimized for the computation-intensive aspects of point cloud kernels. Figure 6 further illustrates the instruction formats. These instructions are designed to work in conjunction with the CPU’s existing capabilities, such as out-of-order execution for control-intensive tasks.²

The instructions are categorized as follows:

²In this work, we assume the presence of matrix multiplication instructions (e.g., via ARM’s SME extension) in the base ISA, so they are not included here to maintain focus on point cloud-specific optimizations.

Distance Computation: The instruction **SEDS** (Sum of Euclidean Distance Squared) computes the Euclidean distance squared between elements in two vectors. This instruction processes one dimension per invocation, providing fine-grained control and flexibility for varying data dimensions, thereby accelerating the core computation in FPS and NN without burdening the CPU with parallel management. **Sorting:** There are three sorting instructions—**HSTWI** (Horizontal Sort with Index), **VSTWI** (Vertical Sort with Index) and **USTWI** (Universal Sort with Index), which perform row-wise, column-wise and global sorting on two-dimensional tensors, respectively. These instructions abstract common aggregation operations (e.g., argmax, min, top-K) into unified sorting primitives, enabling efficient handling of distance-based decisions in point cloud kernels. **Data Movement:** **MOVT** (Move Tile) instruction facilitates efficient data replication and transfer between matrix and vector registers, minimizing overhead in data-intensive operations.

The new instructions are integrated into the LLVM[23]. Furthermore, to facilitate their usage and seamless integration into existing codebases, we have implemented intrinsics for each custom instruction in LLVM, enabling developers to directly utilize them in high-level code. Figure 7 illustrates a code snippet performing kNN search using PointISA intrinsics, alongside the corresponding assembly code. By combining these instructions, PointISA achieves fine-grained fusion of distance computation and sorting, significantly reducing data movement and enhancing parallelism. Specifically, the **SEDS** instruction computes distances between query points and reference points in parallel, after which the **HSTWI** instruction selects neighbor points. The CPU handles irregular control flows, such as iterations over points or early termination conditions. This separation ensures hardware acceleration of computation-intensive tasks, while maintaining flexibility for control-intensive operations via the CPU, ultimately improving overall efficiency.

³In this paper, we use Ta/Tb to denote matrix tile registers for clarity, while the actual ARM SME assembly uses register names such as za0, za1, etc. as generated by LLVM.

```

#define Ta0 0
#define Ta1 1
#define Ta8 8
// K denotes the number of top-K points
svbool_t_topMask = svwhilelt_b32_s32(0, K);
// Q is the number of query points
svbool_t_queryMask = svwhilelt_b32_s32(0, Q);
// load query points
svfloat32x3_t_query_points = svld3_f32(queryMask, (float*)5queries[0]);

for (int i = 0; i < R; i += svcntw()) {
    // R is the number of reference points
    svbool_t_refMask = svwhilelt_b32_s32(i, R);

    svzero_zs(Ta0, Ta8);
    // load reference points
    svfloat32x3_t_ref_points = svld3_f32(refMask, (float*)6references[i]);
    // compute euclidean distance between query points and reference points
    svecldidean_dist_f32(Ta0, queryMask, refMask);
    svecld3_f32(query_points, 0), svecld3_f32(ref_points, 0);
    svecldidean_dist_f32(Ta0, queryMask, refMask);
    svecld3_f32(query_points, 1), svecld3_f32(ref_points, 1);
    svecldidean_dist_f32(Ta0, queryMask, refMask);
    svecld3_f32(query_points, 2), svecld3_f32(ref_points, 2);
    // write indices of reference points to index the topK points after sort
    for (int tileRow = 0; tileRow < svcntw(); tileRow++) {
        swrite_hor_zs32_s32_m(Ta0, tileRow, refMask, svindex_s32(i, 1));
    }
    // sort for topK
    svsort_hor_b32_s32(Ta1, Ta0, topKMask, refMask);
}
//...

```

```

.Block_A:
    whilelt p1.s, wzr, w24
    ldur x9, [x29, #-88]
    ldr x8, [x21]
    whilelt p8.s, wzr, w9
    cmp w22, #1
    ld3w { z0.s - z2.s }, p1/z, [x8]
    b.lt .Block_D
    mov w8, wzr
    cntw x9
    mov w10, #12

.Block_B:
    whilelt p2.s, w8, w22
    zero #17, #17
    ldr x11, [x20]
    smaddl x11, w8, w10, x11
    index z3.s, w8, #1
    mov x12, xzr
    ld3w { z4.s - z6.s }, p2/z, [x11]
    seds za0.q, p1/m, p2/m, z0.s, z4.s
    seds za0.q, p1/m, p2/m, z1.s, z5.s
    seds za0.q, p1/m, p2/m, z2.s, z6.s

.Block_C:
    mova za0h.q[w12, #], p2/m, z3.s
    add x12, x12, #1
    cmp x9, x12
    b.ne .Block_C
    add w8, w8, w9
    mntw za1h.q, za0h.q, p8/m, p2/m
    cmp w8, w22
    b.lt .Block_B
    .Block_D:
    ...

```

(a) C implementation. (b) assembly code by LLVM.

Figure 7: C code snippet for kNN search using PointISA intrinsics and the corresponding generated assembly code.³

5 Versatile Systolic Array for PointISA

5.1 Overview Architecture

To support the execution of *PointISA* instructions and conventional matrix operations, we propose a lightweight and versatile systolic array architecture. By reusing the systolic array structures already integrated in existing matrix extensions (e.g., AMX, SME), our design minimizes additional hardware overhead while ensuring efficient acceleration for point cloud analytics. An overview of the architecture is shown in Figure 8.

The core of the proposed architecture is an 8×8 processing element (PE) array, organized as a two-dimensional mesh. Each PE is interconnected with its neighbors to facilitate data flow across the array. Specifically, each column of PEs is connected end-to-end, receiving input data from the north port and outputting results to the south port. Similarly, each row of PEs is connected end-to-end, with data entering from the west port and exiting through the east port. To enable cyclic data movement, the last PE in each row or column outputs data back to the first PE, forming a ring link. Additionally, to support full sorting functionality, the PE at the beginning of each row receives output from the last PE of the previous row (with the first row receiving data from the last row), ensuring seamless data propagation across the entire array for operations like sorting and aggregation.

A key feature of our design is seamless integration with the standard CPU pipeline. Our architecture directly leverages the ARM SME hardware framework, augmenting each PE by incorporating compact subtractor and comparator units while reusing the existing MAC components. On the instruction level, we introduce minimal, incremental modifications to the CPU's decoder. Since our new instructions adhere to the established ARM SME format, this decoding overhead is negligible. As shown in Figure 8, the array interfaces directly with the CPU's load-store unit and vector registers, ensuring low-latency data transfers. This methodology

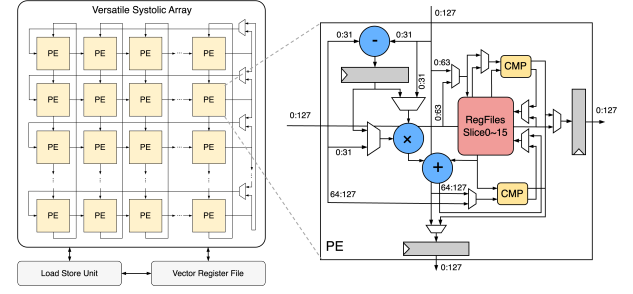


Figure 8: Architecture overview of versatile systolic array and PE design.

balances powerful, specialized acceleration with practical deployability, making it a compatible and efficient extension for real-world systems.

5.2 PE structure

The architecture of the PE is illustrated in Figure 8. Each PE consists of a register file, some arithmetic units, two comparison units, two output registers and some MUXs, supporting matrix storage, square distance calculation and sorting functions. The registers in a single PE are composed of 16 register slices, numbered 0-15. For the entire PE array, register slices with the same number form a 8×8 matrix register tile. Therefore, there are 16 matrix register tiles in total.

When the systolic array executes the *SEDS* instruction, each PE performs squared distance computation under an output-stationary model. The north and west ports input the coordinates of two points in a certain dimension respectively, calculate the square distance through the subtractor and multiplier, and accumulate it with the selected register (by tile ID) in RegFiles through the adder. The accumulated result will be stored back to the same set of registers in RegFiles. The input data can also skip the subtractor to support the vector outer product.

5.3 Hardware Support for Multidimensional Sorting

The versatile systolic array supports three sorting modes: vertical mode, horizontal mode and full mode, respectively. Multiple data paths of the PE array are designed to support data transmission modes in these three sorting modes.

The sorting logic is shown in Figure 9, where the sorting process consists of two stages. In each stage, each PE processes the two selected register values in its RegFiles, compares them with the two input values, and processes them according to the comparison results. Taking the n^{th} PE as an example, in the cycle t of the first stage, the register X in PE^n stores the input E of the previous cycle $t-1$, and the register Y stores the maximum value of X and Y in the previous cycle. At the same time, PE^n also outputs the minimum value of X and Y in the current cycle to E , and sets the output F to 0. When the first stage ends, every value in X will be less than Y , which takes $\frac{N}{2}$ cycles. In cycle t of the second stage, the register X in PE^n stores the minimum value between the input E and X of the present cycle t , while the register Y stores the minimum value

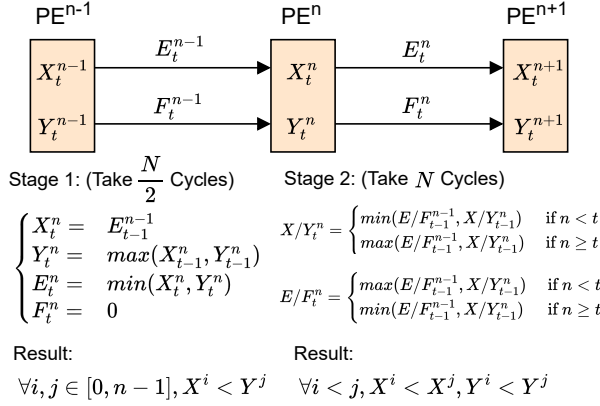


Figure 9: Illustration of the two-stage inter-PE data exchange and comparison for sorting.

of the input F and Y in the previous cycle $t-1$. PE^n outputs the maximum value of E and X in the current cycle to E , and sets the output F to the maximum between F and Y in the previous cycle $t-1$. When the second stage ends, the values in the registers X and Y are arranged from small to large according to the PE index, which takes N cycles. Since the first stage ensures that the values in X are all less than Y , X and Y are completely ordered. In total, $1.5N$ cycles are needed to complete the sequence of N lengths, which achieves a speedup of 33% compared to the traditional systolic sort architecture[48].

6 Algorithm Transformation

The proposed instruction set and hardware architecture enable two-dimensional parallel computation capabilities for point cloud analytics. However, as discussed in Section 3.1, existing point cloud workloads primarily exhibit SP2MP computational parallelism, limiting the utilization of available two-dimensional parallel computing resources. In this section, we propose transformations for the farthest point sampling and k-nearest neighbor kernels to fully leverage the capabilities introduced by the instruction set extensions.

6.1 Lazy Farthest Point Sampling Algorithm

Traditional FPS algorithms necessitate updating the distances from points to the sampled set after each sampling iteration. These updates are essential for the precise identification of the next farthest point. However, such immediate updates are often unnecessary. Given that distances between unsampled points and the sampled set monotonically decrease, points that are significantly closer to the sampled set are unlikely to become sampling candidates in the short term. Thus, updating distances for these points can be safely postponed. Building on this insight, we propose a *lazy* FPS algorithm, wherein each distance computation round yields multiple sampled points rather than a singular one. Within this framework, the computational workload transitions from SP2MP calculations to an MP2MP manner, effectively leveraging the proposed PointISA extensions.

Algorithm 1: Lazy KD Tree-based Farthest Point Sampling

Input : Points P ; Sample number N ; Node number M ; Vector length V

Output : Sampled result S ;

- 1 Initialization: $S = \{P_0\}$ and $cnt = 1$;
- 2 $B \leftarrow$ Build KD tree of P with M Node;
- 3 Initialize $farPoint$ and $secondFarPoint$ for every Node;
- 4 **while** $cnt < N$ **do**
- // USTWI for top2
- 5 $[FarNode, SecNode] \leftarrow \text{top2}(B.farPoint.dist)$;
- 6 **if** $dist(FarNode.farPoint, S) > SecNode.farPoint.dist$ **then**
- 7 $S.pushback(FarNode.farPoint), cnt++$;
- 8 $FarNode.farPoint \leftarrow FarNode.secondFarPoint$;
- 9 $Invalid FarNode.secondFarPoint$;
- 10 **else**
- 11 $Invalid FarNode.farPoint$;
- 12 **if** $FarNode.secondFarPoint.isValid$ **then**
- 13 $FarNode.farPoint \leftarrow FarNode.secondFarPoint$;
- 14 $Invalid FarNode.secondFarPoint$;
- 15 **else**
- // loop unrolling, factor= V
- 16 **foreach** unprocessed & unprunable point s in S **do**
- // loop unrolling, factor= V
- 17 **foreach** point p in $FarNode.points$ **do**
- // SEDS and HSTWI for $dist$ and min
- 18 $p.dist \leftarrow \min(p.dist, dist(p, s))$;
- 19 $FarNode.sampleStamp \leftarrow cnt$;
- // using USTWI to find farPoint and secondFarPoint
- 20 update $farPoint$ and $secondFarPoint$ of $FarNode$;

Algorithm 1 illustrates the pseudocode for the lazy KD Tree-based FPS algorithm. Initially, the point cloud is partitioned into multiple nodes using a KD tree, with each node containing a subset of unsampled points. By sorting distances within each node, the algorithm identifies both the farthest and the second-farthest points to the current sampled set. The second-farthest point provides a critical reference for distance validation. During each sampling iteration, the farthest and second-farthest nodes are selected based on their current maximum distances, employing the USTWI instruction. As shown in Line 6-9, since the distance of the farthest node may be outdated, the distance between the farthest point within this node and the sampled set is recalculated. If this updated distance remains greater than that of the second-farthest node, the farthest point is selected as the new sampled point. Otherwise, the second-farthest point becomes a temporary candidate, initiating the next iteration. As shown in Line 16-20, when both the farthest and second-farthest points of a node become outdated, the node's state is refreshed through highly parallel distance updates using SEDS and HSTWI instructions.

Figure 10 compares traditional FPS and the proposed algorithm. By identifying and deferring unnecessary distance updates, our

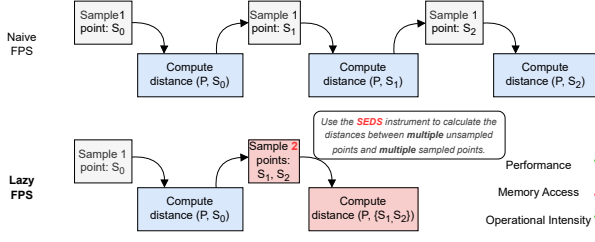


Figure 10: An example of applying lazy computation method. Compared to native FPS, it generates multiple sampled points between the distance computing stages, which reduce the memory access and improve the operational intensity.

Algorithm 2: Lazy KD Tree-based kNN Search

```

Input : Query Points  $Q$ ; Reference Points  $P$ ; Node number  $M$ ;
        Topk  $k$ ; Vector length  $V$ 
Output: K-Nearest Neighbor Points  $S$ 
1  $B \leftarrow$  Build KD tree of  $P$  with  $M$  Node;
2 Dispatch all query points into the nearest Node's queryList;
3 while !all  $M$ .queryList is empty do
4   foreach node  $n$  in  $B$  do
      // loop unrolling, factor=  $V$ 
5     foreach point  $q$  in  $n$ .queryList do
      // loop unrolling, factor=  $V$ 
6       foreach point  $p$  in  $n$ .points do
          // SEDS and HSTWI for dist and sort
7          $q$ .curTopK  $\leftarrow$   $\text{sort}(q$ .curTopK,  $\text{dist}(q, p))$ 
8          $\text{nextNode} \leftarrow q$ .nextTravelNode();
9         if  $q$ .curTopK[ $k$ ] >  $\text{boundDist}(q, \text{nextNode}.box)$ 
            then
10           $\text{nextNode}.queryList.push\_back(q)$ ;
11    $n$ .queryList.clean();

```

approach generates multiple sampled points per iteration without affecting sampling accuracy, thus substantially increasing the operational intensity and reducing memory access.

6.2 Lazy K-Nearest Neighbor Search Algorithm

In conventional kNN search algorithms, the irregular KD tree traversals that are caused by spatially random query distributions severely limit the efficiency of query-level parallelism. To address this challenge, we introduce a *lazy* kNN search algorithm. Specifically, a cache queue is introduced at each leaf node, and the traversal of a single query point across multiple KD tree nodes is partitioned into distinct stages. Each stage focuses exclusively on nearest neighbor searches within a single node. By transforming temporal irregularities of multiple queries into spatial regularities, we achieve to construct MP2MP computation tasks, effectively utilizing the proposed ISA extensions.

Algorithm 2 presents the pseudocode for the lazy kNN search method. Initially, the point cloud is divided into multiple nodes using a KD tree structure. All query points are distributed across nodes based on spatial locality, resulting in each node containing

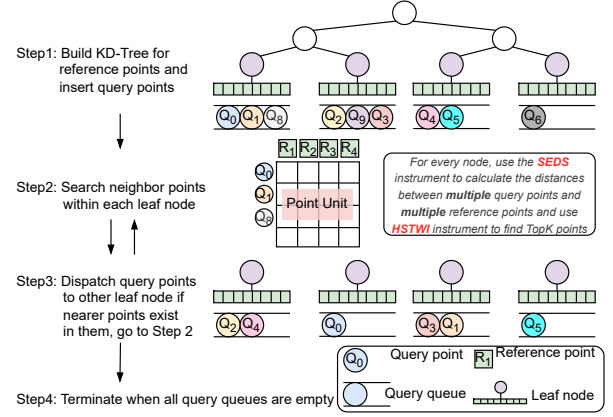


Figure 11: An example of applying lazy computation method. Compared to the native kNN, it generates MP2MP tasks within each node.

multiple query points alongside reference points. Consequently, the highly parallel computation of nearest neighbors can be executed using the SEDS and HSTWI instructions within each node. However, potential nearest neighbors may reside in multiple nodes. Therefore, based on intermediate search results and traversal paths, each query point is scheduled for the next node traversal by appending it to that node's query queue. After processing all nodes, the algorithm terminates if all node queues are empty; otherwise, the next traversal round begins.

Figure 11 illustrates the computational process of the proposed algorithm. By decomposing and reorganizing query point traversals across multiple nodes, our approach aggregates computations for multiple query and reference points without impacting search accuracy, significantly enhancing computational parallelism.

7 Experiment

7.1 Experimental Setup

Benchmarks. We have conducted a comprehensive end-to-end evaluation of PointISA across diverse point cloud applications, as detailed in Table 4. For classification, we use PointNet++(c)[35] and Point Transformer[49] on the ModelNet40[43] dataset. Both PointNet++(c) and Point Transformer involve geometry-based FPS and radius search operations. For segmentation, we use DGCNN[41] on the S3DIS[1] dataset and PointNeXt[36] on the ScanNet[6] dataset. The DGCNN workload contains both geometry-based and feature-based kNN operations. The PointNeXt workload involve geometry-based FPS and radius search operations. For object detection, we use 3DSSD[44] in the KITTI[14] dataset. 3DSSD are the first model to import feature-based FPS into point cloud analytics. We also use the SiMPLE[2] algorithm on the KITTI dataset to evaluate PointISA on conventional point cloud-based workloads. Such extensive benchmarks allow us to thoroughly evaluate PointISA.

Hardware Implementation. We have implemented PointISA in the gem5 simulator, targeting an Out-of-Order (OoO) CPU based on ARM's AArch64 ISA. While our solution is ISA-agnostic, we

Table 3: Baseline CPU Model.

Parameter	Value
CPU	OoO ARM v9 64-bit @3GHz, Fetch Width: 3, Issue Width: 8, Int Physical Reg.: 90, Float/Vector Physical Reg.: 256, Mat Physical Reg.: 4, SVE: 1.0 256-bit, SME: 1.0 256-bit
Memory	L1: 32KB (I) 2-way + 32KB (D) 2-way, L2: 1MB
System	16-way, Main Memory: 8GB DDR4-2400

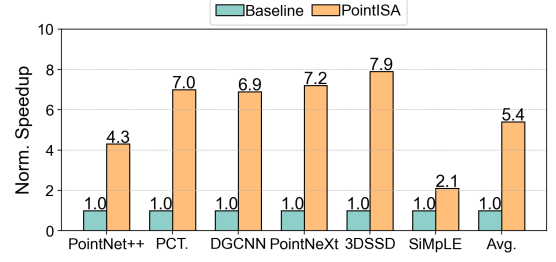
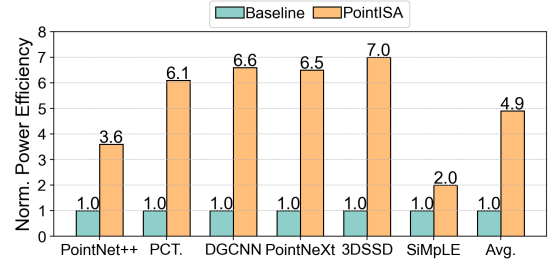
Table 4: Evaluation Benchmarks.

Application Domains	Algorithm	Dataset	Key Operations
Classification	PointNet++(c) Point Transformer	ModelNet40	G-FPS, G-RNN GEMM
Segmentation	DGCNN	S3DIS	G-KNN, F-KNN GEMM
	PointNeXt(b)	ScanNet	G-FPS, G-RNN GEMM
Object Detection	3DSSD	KITTI	G-FPS, F-FPS G-RNN, GEMM
Registration	SiMPL	KITTI	G-KNN, G-RNN

G-xxx = Geometry-based operations; F-xxx = Feature-based operations.
 "Point Transformer" is abbreviated as PCT.

use ARM as a representative architecture for point cloud analytics due to its widespread adoption in embedded platforms. Our model is based on the pre-configured gem5 *big* CPU core, with detailed parameters listed in Table 3. To accurately reflect hardware characteristics, we calibrated the execution latency of each PointISA instruction in gem5 according to hardware implementation results, with the systolic array operating at 750 MHz. To ensure a fair comparison, both the baseline and our PointISA-enhanced model support the ARM SVE and SME extensions. Since no existing libraries optimize point cloud operations (FPS/kNN) for ARM SVE and SME extensions, we develop self-optimized kernels based on the Point Cloud Library (PCL)[38] reference implementation. We also apply software optimizations such as prefetching, loop unrolling and vectorization to maximize baseline performance under SVE/SME. All benchmarks are compiled with `-O3` optimization. This ensures that performance improvements only come from PointISA's architectural enhancements rather than compiler bias.

Experimental Methodology. We implement the systolic array using Verilog and verify the design through RTL simulations. Power consumption is estimated using Synopsys Design Compiler with a 28 nm process technology. Area estimation is performed using Synopsys IC Compiler for floorplanning, followed by IC Compiler II for placement and routing. For energy analysis, we model the CPU in McPAT [25] and utilize statistics reported by gem5 to drive the McPAT power model.

**Figure 12: Performance improvement over the baselines on six workloads.****Figure 13: Power efficiency improvement over the baselines on six workloads.**

7.2 Performance and Power Efficiency

The experimental results demonstrate significant improvements in both performance and power efficiency across all evaluated workloads. As shown in Figure 12, PointISA delivers a geometric mean speedup of 5.4× compared to the baseline implementation. Correspondingly, Figure 13 reveals remarkable power efficiency, with a geometric mean improvement of 4.9× across these workloads. These improvements primarily stem from two key architectural advantages: (1) the efficient hardware support for critical point cloud operations in PointISA, and (2) reduced data movement overhead between registers and L1 cache through optimized matrix register operations. Notably, the acceleration benefits are more pronounced for point cloud neural networks (4.3–7.9×) than for registration tasks (2.1×), as the latter's performance is fundamentally constrained by neighbor search operations which account for approximately 40% of the computation time and thus limit the maximum achievable optimization.

We further conduct in-depth analysis of the architectural mechanisms underlying PointISA's performance gains, as quantitatively demonstrated in Figure 14. The extended instruction set reduces the number of committed instructions by 89%, enabled by fused operations for distance computation and sorting. L1 cache accesses also decrease significantly. This is primarily due to two factors: (1) the fine-grained fusion of Euclidean distance computation and sorting instructions eliminates the intermediate data movements between matrix registers and L1 Cache. and (2) the MP2MP computation pattern enhances data reuse locality. In contrast, L2 and DRAM access patterns remain largely unchanged. This is because the global dependencies in point cloud workloads often exceed the

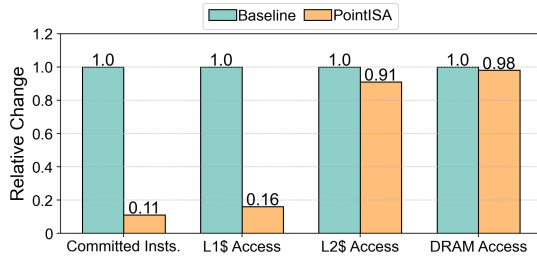


Figure 14: Normalized hardware metrics for the PointNet++ workload.

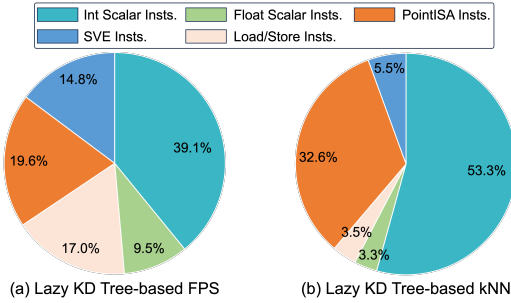


Figure 15: The instruction breakdown of FPS and kNN workload on S3DIS dataset.

capacity of L1 cache. We leave a more thorough exploration of memory hierarchy optimization to future work.

To further understand PointISA’s runtime behavior, we have analyzed the instruction breakdown during FPS and kNN execution. As shown in Figure 15. PointISA instructions account for 19.6% of total instructions in FPS and 32.6% in kNN, confirming their higher relevance in compute-intensive stages.

7.3 Area and Power Analysis

The versatile systolic array is modeled using a 28nm technology library. Figure 16 depicts the PnR ASIC layout. A folded-torus layout strategy is employed to alleviate backend routing pressure introduced by the long wrap-around links of 2D torus topology. The total area required is 0.547 mm² and the detailed area breakdown is also presented. The computing unit within each PE constitutes the primary area consumer, accounting for approximately 56.7% of the total footprint. When compared to an ARM Cortex-A72 processor (including L2 cache)⁴, the proposed systolic array represents only 0.9% of the reference processor’s area. Power analysis reveals a 340.8 mW power consumption for the systolic array during typical operation. This modest area and power overhead not only demonstrates PointISA’s efficiency in point cloud processing but also supports its long-term viability by minimizing hardware costs, enabling integration into resource-constrained SoC designs without significant legacy burdens.

⁴Baseline CPU area is scaled to 28nm using McPAT data and the scaling methodology proposed by Stillmaker et al.[40] for fair comparison

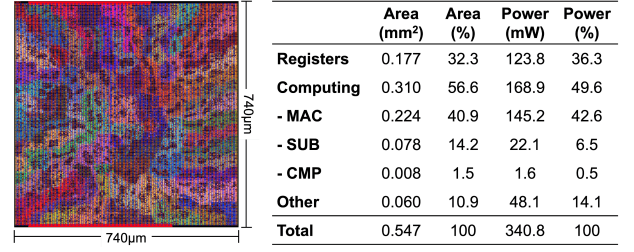


Figure 16: Versatile systolic array floorplan with its area and power breakdown.

7.4 Sensitivity Analysis

To systematically evaluate the performance benefits from both algorithmic and architectural optimizations, we conduct a comprehensive sensitivity analysis on two fundamental point cloud operators - FPS and kNN - across four configurations (Baseline, Only PointISA, Only Algorithm and PointISA+Algorithm) using the S3DIS dataset. As illustrated in Figure 17, the synergistic combination of hardware and software optimizations achieves remarkable speedups of 13.3× for FPS and 27.84× for kNN. Individual optimizations, however, exhibit distinct characteristics. FPS benefits modestly from PointISA alone due to its inherent SP2MP computation pattern’s limited parallelism. In contrast, kNN gains only 1.42× from algorithmic optimization alone because SVE cannot effectively exploit the MP2MP pattern created by our lazy kNN algorithm. These results demonstrate the critical importance of hardware-software co-design in point cloud processing. Even with theoretically superior algorithms, performance gains remain severely limited without appropriate hardware support. In fact, our analysis reveals that PointISA instructions deliver over an order of magnitude performance improvement compared to equivalent SVE instructions for these workloads, but this potential remains largely untapped when using algorithm optimization alone with conventional hardware.

We further evaluate the performance of PointISA on FPS and kNN with different point cloud sizes. As shown in Figure 18, the number of points increases from 1,024 to 32,768, covering most common scales in point cloud workloads. As the scale grows, both the total runtime and per-sample point time of FPS increase faster than those of kNN. This is because FPS involves more irregular memory access. At small scales, most data can be handled by the L1 cache. But as the point cloud size increases, irregular access leads to more frequent L2 and DRAM accesses, causing higher latency.

7.5 Comparison with the existing accelerator

To better understand the trade-offs between general-purpose and specialized designs, we compare PointISA with PointAcc, a domain-specific accelerator for point cloud workloads. As shown in Figure 19, PointISA is 17% slower than PointAcc. This is mainly because PointISA executes GEMM operations less efficiently than PointAcc. However, PointISA outperforms PointAcc on the PointNeXt workload with large-scale dataset. This is because PointAcc only supports brute-force implementations of FPS and kNN to reduce control complexity, while PointISA’s flexibility enables the use of more efficient algorithmic optimizations.

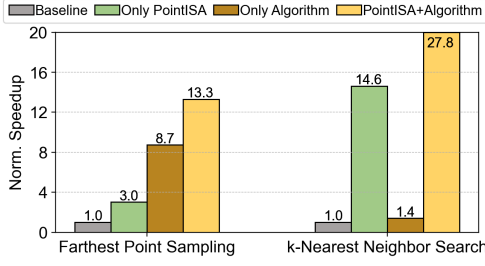


Figure 17: Normalized performance gained with different design points of FPS and kNN workload on S3DIS dataset.

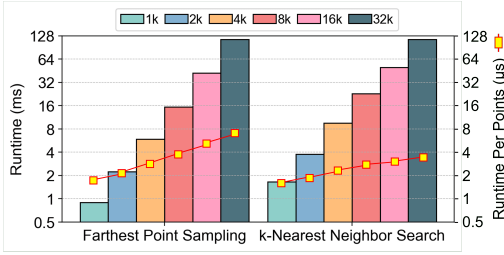


Figure 18: Runtime of the FPS and kNN with several scales of point clouds. The sample rate is 50% for FPS and the k is 20 for kNN. The red line shows the average runtime per sampled (for FPS) or per query (for kNN) point at each scale.

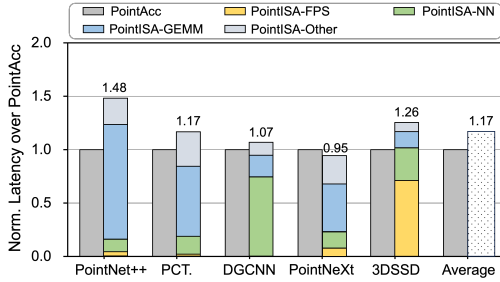


Figure 19: Normalized latency for various neural network workloads, and the latency breakdown of PointISA.

In terms of area efficiency, PointISA reuses existing CPU resources and adds only about 0.547 mm^2 of overhead. In contrast, PointAcc requires 1.9 mm^2 of dedicated hardware, resulting in a higher area-delay product (ADP) than PointISA. Moreover, since point cloud analytics is only a part of heterogeneous workload in real word embedded system, ISA extensions offer a more cost-effective compared to dedicated accelerators, leading to better resource utilization in SoC designs.

In terms of programmability, PointISA supports high-level coding with LLVM intrinsics. This enables flexible C/C++ development. This programmability provides a solid foundation for further optimizations, such as parallel execution on multicore systems and operator fusion at the software level. Additionally, PointISA can easily adapt to future algorithmic innovations, as new optimization techniques can be implemented through software updates without hardware modifications.

8 Related Works

Specific Kernel Acceleration for Point Cloud. In point cloud processing, specific kernels like neighbor searching and farthest point sampling often become bottlenecks, prompting targeted acceleration efforts. For neighbor searching, partition-based accelerators such as QuickNN [32] and ParallelNN [5] split point clouds into buckets using KD trees, octrees, or voxels, then apply approximate search algorithms to reduce the search space. Additionally, K-D Bonsai [8] proposes an ISA extension for point cloud compression to optimize memory usage in radius search, while BitNN [16] leverages bit-level parallelism and early termination for performance gains via bit-serial computing. For farthest point sampling, QuickFPS [18] and FuseFPS [17] employ partition-based methods and KD tree fusion to reduce computation and memory access. However, these approaches focus on specific kernels and fail to support end-to-end inference acceleration for point cloud analytics.

Domain-Specific Accelerators for Point Cloud Neural Networks. Point cloud neural networks, a key area in 3D vision, have inspired various accelerator designs. Mesorasi [12] uses delayed aggregation, while PointAcc [26] maps diverse kernels to sorting-based units, exploiting sparse computing and layer fusion for high performance. Similarly, Point-X and GDPCA [4] enhance efficiency by leveraging spatial locality and clustering in graph-based networks. Crescent addresses memory irregularities, and accelerators like EdgePC [45] and HgPCN [13] use Morton coding and octrees to structure point clouds into subspaces, optimizing sampling and search complexity. SPADE [24] focuses on bird’s-eye-view networks, benefiting from pillar sparsity. Nevertheless, most accelerators adopt loosely coupled designs with independent execution units for neighbor search, FPS and GEMM, leading to uneven load distribution and low utilization. In contrast, PointISA implements a highly coupled systolic array that uniformly supports Euclidean distance calculations, multidimensional sorting and matrix operations, achieving higher device utilization.

9 Conclusion

This paper presents PointISA, a novel instruction set architecture extension for point cloud analytics through architecture-algorithm co-design. Our approach introduces specialized Euclidean distance and sorting instructions, implements a versatile systolic array architecture, and reconstructs key algorithms to create optimized MP2MP computation patterns. Experimental results show $5.4\times$ average speedup and $4.9\times$ power efficiency improvement with minimal (0.9%) area overhead.

In further work, we plan to explore more efficient memory architectures and memory management techniques[28, 29] to address the inefficiencies caused by irregular memory access patterns in point cloud workloads and we will also investigate the software optimizations, such as multicore parallelization, operator fusion techniques and resource scheduling[27] to further improve performance of point cloud workloads.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (Grant No. 62204139, No. U22B2024), and in part by No.31513010104.

References

- [1] Iro Armeni, Ozan Sener, Amir R Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 2016. 3d semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 1534–1543.
- [2] Vedant Bhandari, Tyson Govan Phillips, and Peter Ross McAree. 2024. Minimal configuration point cloud odometry and mapping. *The International Journal of Robotics Research* 43, 11 (2024), 1831–1850.
- [3] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. 2011. The gem5 simulator. *ACM SIGARCH computer architecture news* 39, 2 (2011), 1–7.
- [4] Cen Chen, Xiaofeng Zou, Hongen Shao, Yangfan Li, and Kenli Li. 2023. Point cloud acceleration by exploiting geometric similarity. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1135–1147.
- [5] Faquan Chen, Rendong Ying, Jianwei Xue, Fei Wen, and Peilin Liu. 2023. Parallelnn: A parallel octree-based nearest neighbor search accelerator for 3d point clouds. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 403–414.
- [6] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. 2017. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 5828–5839.
- [7] Lei Dai, Shengwen Liang, Ying Wang, Huawei Li, and Xiaowei Li. 2024. APoX: Accelerate Graph-Based Deep Point Cloud Analysis via Adaptive Graph Construction. In *Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 231–237.
- [8] Pedro H E Becker, José-Maria Arnaiz, and Antonio González. 2023. KD bonsai: ISA-extensions to compress KD trees for autonomous driving tasks. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [9] Elisa Mariarosaria Farella, Alessandro Torresani, and Fabio Remondino. 2019. Sparse point cloud filtering based on covariance features. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* 42 (2019), 465–472.
- [10] Yu Feng, Gunnar Hammonds, Yiming Gan, and Yuhao Zhu. 2022. Crescent: taming memory irregularities for accelerating deep point cloud analytics. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 962–977.
- [11] Yu Feng, Zheng Liu, Weikai Lin, Zihan Liu, Jingwen Leng, Minyi Guo, Zhezhi He, Jieru Zhao, and Yuhao Zhu. 2025. StreamGrid: Streaming Point Cloud Analytics via Compulsory Splitting and Deterministic Termination. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1189–1202.
- [12] Yu Feng, Boyuan Tian, Tiancheng Xu, Paul Whatmough, and Yuhao Zhu. 2020. Mesorasi: Architecture support for point cloud analytics via delayed-aggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1037–1050.
- [13] Yiming Gao, Chao Jiang, Wesley Piard, Xiangru Chen, Bhavesh Patel, and Herman Lam. 2024. HgPCN: A Heterogeneous Architecture for E2E Embedded Point Cloud Inference. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1588–1600.
- [14] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. IEEE, 3354–3361.
- [15] Yulan Guo, Hanyun Wang, Qingyong Hu, Hao Liu, Li Liu, and Mohammed Bennamoun. 2020. Deep learning for 3d point clouds: A survey. *IEEE transactions on pattern analysis and machine intelligence* 43, 12 (2020), 4338–4364.
- [16] Meng Han, Liang Wang, Limin Xiao, Hao Zhang, Tianhao Cai, Jiale Xu, Yibo Wu, Chenhao Zhang, and Xiangrong Xu. 2024. BitNN: A Bit-Serial Accelerator for K-Nearest Neighbor Search in Point Clouds. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1278–1292.
- [17] Meng Han, Liang Wang, Limin Xiao, Hao Zhang, Chenhao Zhang, Xilong Xie, Shuai Zheng, and Jin Dong. 2024. FuseFPS: Accelerating Farthest Point Sampling with Fusing KD-tree Construction for Point Clouds. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 238–243.
- [18] Meng Han, Liang Wang, Limin Xiao, Hao Zhang, Chenhao Zhang, Xiangrong Xu, and Jianfeng Zhu. 2023. QuickFPS: Architecture and algorithm co-design for farthest point sampling in large-scale point clouds. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 11 (2023), 4011–4024.
- [19] Geonhwa Jeong, Eric Qin, Ananda Samajdar, Christopher J Hughes, Sreenivas Subramoney, Hyesoon Kim, and Tushar Krishna. 2021. Rasa: Efficient register-aware systolic array matrix engine for cpu. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 253–258.
- [20] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [21] Shinpei Kato, Eijiro Takeuchi, Yoshio Ishiguro, Yoshiki Ninomiya, Kazuya Takeda, and Tsuyoshi Hamada. 2015. An open approach to autonomous vehicles. *IEEE Micro* 35, 6 (2015), 60–68.
- [22] Hyungyo Kim, Gaoan Ye, Nachuan Wang, Amir Yazdanbakhsh, and Nam Sung Kim. 2024. Exploiting intel® advanced matrix extensions (amx) for large language model inference. *IEEE Computer Architecture Letters* (2024).
- [23] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International symposium on code generation and optimization*. IEEE, 75–86.
- [24] Minjae Lee, Seongmin Park, Hyungmin Kim, Minyong Yoon, Janghwan Lee, Jun Won Choi, Nam Sung Kim, Mingu Kang, and Jungwook Choi. 2024. Spade: Sparse pillar-based 3d object detection accelerator for autonomous driving. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 454–467.
- [25] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd annual IEEE/ACM international symposium on microarchitecture (MICRO)*. 469–480.
- [26] Yujun Lin, Zhekai Zhang, Haotian Tang, Hanrui Wang, and Song Han. 2021. Pointacc: Efficient point cloud accelerator. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 449–461.
- [27] Lei Liu, Xinglei Dou, and Yuetao Chen. 2023. Intelligent Resource Scheduling for Co-located Latency-critical Services: A {Multi-Model} Collaborative Learning Approach. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 153–166.
- [28] Lei Liu, Yong Li, Chen Ding, Hao Yang, and Chengyong Wu. 2015. Rethinking memory management in modern operating system: Horizontal, vertical or random? *IEEE Trans. Comput.* 65, 6 (2015), 1921–1935.
- [29] Lei Liu, Shengjie Yang, Lu Peng, and Xinyu Li. 2019. Hierarchical hybrid memory management in OS for tiered memory systems. *IEEE Transactions on Parallel and Distributed Systems* 30, 10 (2019), 2223–2236.
- [30] Ming Liu. 2015. Robotic online path planning on point cloud. *IEEE transactions on cybernetics* 46, 5 (2015), 1217–1228.
- [31] Xueyan Liu, Zhuoran Song, Guohao Dai, Gang Li, Can Xiao, Yan Xiang, Dehui Kong, Ke Xu, and Xiaoyao Liang. 2024. FusionArch: A Fusion-Based Accelerator for Point-Based Point Cloud Neural Networks. In *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [32] Reid Pinkham, Shuqing Zeng, and Zhengya Zhang. 2020. Quicknn: Memory and performance optimization of kd tree based nearest neighbor search for 3d point clouds. In *2020 IEEE International symposium on high performance computer architecture (HPCA)*. IEEE, 180–192.
- [33] Allan Pinkus. 1999. Approximation theory of the MLP model in neural networks. *Acta numerica* 8 (1999), 143–195.
- [34] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 652–660.
- [35] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. 2017. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in neural information processing systems* 30 (2017).
- [36] Guocheng Qian, Yuchen Li, Houwen Peng, Jinjie Mai, Hasan Hammoud, Mohamed Elhoseiny, and Bernard Ghanem. 2022. Pointnext: Revisiting pointnet++ with improved training and scaling strategies. *Advances in neural information processing systems* 35 (2022), 23192–23204.
- [37] Thinal Raj, Fazida Hanim Hashim, Aqilah Baseri Huddin, Mohd Faisal Ibrahim, and Aini Hussain. 2020. A survey on LiDAR scanning mechanisms. *Electronics* 9, 5 (2020), 741.
- [38] Radu Bogdan Rusu and Steve Cousins. 2011. 3d is here: Point cloud library (pcl). In *2011 IEEE international conference on robotics and automation*. IEEE, 1–4.
- [39] Shaoshuai Shi, Xiaogang Wang, and Hongsheng Li. 2019. Pointtrnn: 3d object proposal generation and detection from point cloud. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. 770–779.
- [40] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.
- [41] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. 2019. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (tog)* 38, 5 (2019), 1–12.
- [42] Finn Wilkinson and Simon McIntosh-Smith. 2022. An initial evaluation of arm’s scalable matrix extension. In *Proceedings of the IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. IEEE, 135–140.
- [43] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 1912–1920.
- [44] Zetong Yang, Yanan Sun, Shu Liu, and Jiaya Jia. 2020. 3dssd: Point-based 3d single stage object detector. In *Proceedings of the IEEE/CVF conference on computer*

- vision and pattern recognition (CVPR)*. 11040–11048.
- [45] Ziyu Ying, Sandeepa Bhuyan, Yan Kang, Yingtian Zhang, Mahmut T Kandemir, and Chita R Das. 2023. EdgePC: Efficient deep learning analytics for point clouds on edge devices. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–14.
- [46] Ziyu Ying, Shulin Zhao, Sandeepa Bhuyan, Cyan Subhra Mishra, Mahmut T Kandemir, and Chita R Das. 2022. Pushing point cloud compression to the edge. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 282–299.
- [47] Jie-Fang Zhang and Zhengya Zhang. 2021. Point-x: A spatial-locality-aware architecture for energy-efficient graph-based point-cloud deep learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1078–1090.
- [48] Yanjun Zhang and Si-Qing Zheng. 1995. Design and analysis of a systolic sorting architecture. In *Proceedings of the 7th IEEE Symposium on Parallel and Distributed Processing*. IEEE, 652–659.
- [49] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip HS Torr, and Vladlen Koltun. 2021. Point transformer. In *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*. 16259–16268.
- [50] Shulin Zhao, Haibo Zhang, Cyan Subhra Mishra, Sandeepa Bhuyan, Ziyu Ying, Mahmut Taylan Kandemir, Anand Sivasubramaniam, and Chita Das. 2021. Holoar: On-the-fly optimization of 3d holographic processing for augmented reality. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 494–506.